Language Technology

Chapter 9: Counting Words

https://link.springer.com/chapter/10.1007/978-3-031-57549-5_9

Pierre Nugues

Pierre.Nugues@cs.lth.se

September 7, 2026



Structure of this Lecture

This lecture consists of three parts:

1. Word segmentation and word counting
2. Word and document representation
3. Document classification



Text Segmentation



Latin inscriptions on the *Lapis Niger*. *Corpus Inscriptionum Latinarum*, CIL I, 1. Image from Wikipedia.



Getting the Words from a Text: Tokenization

Tokenization consists in converting a sequence of characters:

`[l, i, s, t, ' ', o, f, ' ', c, h, a, r, a, c, t, e, r, s]`

into a sequence of words:

`[list, of, characters]`

This process can be tricky in practice:

- Dates: 28/02/96
- Numbers: 9,812.345 (English), 9 812,345 (French and German), 9.812,345 (older French convention)
- Abbreviations: km/h, m.p.h.
- Acronyms: S.N.C.F.

Tokenizers rely either on rules (or regular expressions) or on statistical methods.



Tokenizing in Python: Using the Content

Simple program: extracting words

```
import regex as re
re.findall(r'\p{L}+', text)
```

Words and the rest

```
re.findall(r'\p{L}+|[^\s\p{L}]+', text)
```

Numbers and punctuation

```
re.findall(r'\p{L}+|\p{N}+|\p{P}|[^\s\p{L}\p{N}\p{P}]+', text)
```



Tokenizing in Python: Using Boundaries

Simple program

```
pattern1 = r'\s+'
re.split(pattern1, text)
```

Handling punctuation

```
pattern2 = r'([\p{S}\p{P}]+)'
re.split(
    pattern1,
    re.sub(pattern2, r' \1 ', text))
```



Improving Tokenization

A purely word-based tokenization strategy defines what counts as content.
It often fails on nomenclatures such as Item #N23-SW32A, dates, or complex numbers.
A better approach is boundary-based: using whitespace (e.g. `\s`) and punctuation as separators.
However, punctuation marks such as commas, periods, or dashes can also form part of tokens.
Further improvements are possible with microgrammars.
At some point a dictionary becomes necessary:
Can't → can n't, *we'll* → we 'll
J'aime → j' aime, but *aujourd'hui* should remain a single token.
See: `https://github.com/karpathy/minGPT/blob/master/mingpt/bpe.py`



Sentence Segmentation

As with tokenization, sentence segmenters rely either on rules (or regular expressions) or on statistical methods.
Grefenstette and Tapanainen (1994) used the Brown corpus and experimented with increasingly complex rules.
The simplest rule—treating a period as a sentence boundary—achieved 93.20% correct segmentation.
Recognizing numbers improved the results:

`[0-9]+(\/[0-9]+)+`	Fractions, dates
`([+\-])?[0-9]+(\.)?[0-9]*%`	Percentages
`([0-9]+,?)+(\.[0-9]+[0-9]+)*|`	Decimal numbers

Accuracy rose to 93.78%.



Abbreviations

Common patterns (Grefenstette and Tapanainen, 1994):

- Single capitals: *A.*, *B.*, *C.*
- Letters and periods: *U.S.*, *i.e.*, *m.p.h.*
- Capital letter followed by a sequence of consonants: *Mr.*, *St.*, *Assn.*

Regex	**Correct**	**Errors**	**Full stop**
`[A-Za-z]\.`	1,327	52	14
`[A-Za-z]\.([A-Za-z0-9]\.)+`	570	0	66
`[A-Z][bcdfghj-np-tvxz]+\.`	1,938	44	26
Totals	3,835	96	106

Correct segmentation increased to 97.66%.
With an abbreviation dictionary the figure reached 99.07%.



Counting Words with Unix Tools

1. `tr -cs 'A-Za-z' '\n' <input_file |`
   Tokenizes the text in `input_file`. The command `tr` behaves similarly to the Perl `tr`. The result is one word per line, which is passed to the next command.
2. `tr 'A-Z' 'a-z' |`
   Converts uppercase letters to lowercase and passes the output to the next command.
3. `sort |`
   Sorts the words so that identical words appear on adjacent lines.
4. `uniq -c |`
   Removes repeated lines. Adjacent identical lines are replaced by a single line preceded by the count of occurrences (`-c`).
5. `sort -rn |`
   Sorts in reverse (`-r`) numeric (`-n`) order so that the most frequent words appear first.
6. `head -5`



Counting Words in Python

```
def tokenize(text):
    words = re.findall(r'\p{L}+', text)
    return words

def count_unigrams(words):
    frequency = {}
    for word in words:
        if word in frequency:
            frequency[word] += 1
        else:
            frequency[word] = 1
    return frequency
```



Counting Words in Python (continued)

```
if __name__ == '__main__':
    text = sys.stdin.read().lower()
    words = tokenize(text)
    frequency = count_unigrams(words)
    for word in sorted(frequency.keys()):
        print(word, '\t', frequency[word])
```



The Counter Class

Python provides a built-in class for counting items in a list:

```
from collections import Counter
words = tokenize(text)
counter = Counter(words)
```

A `Counter` object behaves like a dictionary:

```
>>> counter['hector']
480
```

and offers a convenient method for sorting frequencies:

```
>>> counter.most_common(10)
[('the', 9948),
 ('and', 6624),
 ('of', 5606),
 ('to', 3329), ...]
```



Demo

Tokenizing and counting:
https://github.com/pnugues/pnlp/tree/main/notebooks



Posting Lists

Many websites, such as Wikipedia, index text with an inverted index. Each word in the dictionary is linked to a posting list that records all the documents in which the word occurs, together with its positions inside those documents.

Collection

D1: Chrysler plans new investments in Latin America.

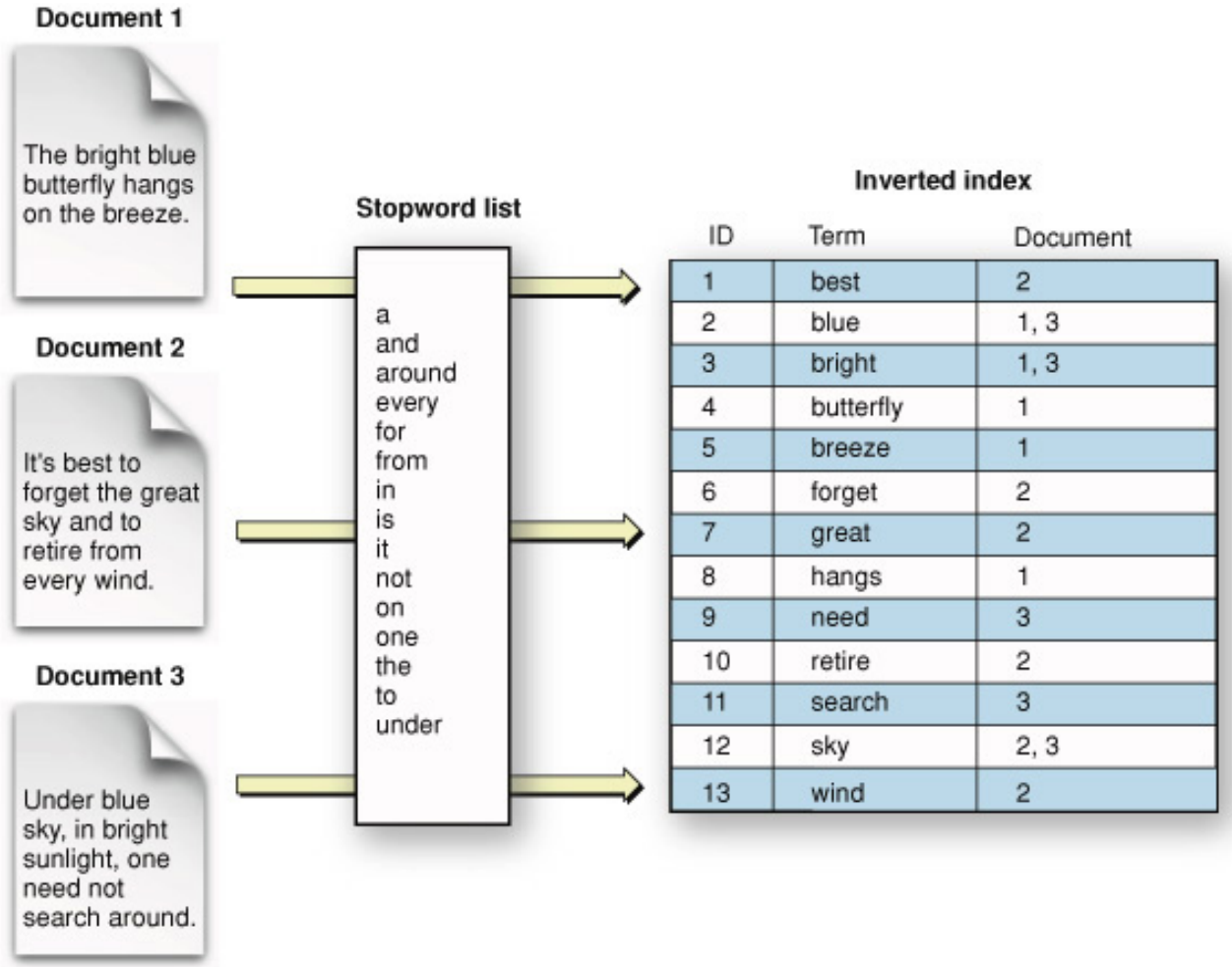
D2: Chrysler plans major investments in Mexico.

Index

Words	Posting lists
America	(D1, 7)
Chrysler	(D1, 1) → (D2, 1)
in	(D1, 5) → (D2, 5)
investments	(D1, 4) → (D2, 4)
Latin	(D1, 6)
major	(D2, 3)
Mexico	(D2, 6)
new	(D1, 3)
plans	(D1, 2) → (D2, 2)

Lucene is a high-quality open-source indexer:

Inverted Index (Source: Apple)



https://developer.apple.com/library/archive/documentation/UserExperience/Conceptual/SearchKitConcepts/searchKit_basics/searchKit_basics.html



Information Retrieval: The Vector Space Model

The vector space model represents each document as a vector in a space whose dimensions are words.

Documents \ **Words**	w_1	w_2	w_3	...	w_m
D_1	$C(w_1, D_1)$	$C(w_2, D_1)$	$C(w_3, D_1)$	...	$C(w_m, D_1)$
D_2	$C(w_1, D_2)$	$C(w_2, D_2)$	$C(w_3, D_2)$	...	$C(w_m, D_2)$
...					
D_n	$C(w_1, D_n)$	$C(w_2, D_n)$	$C(w_3, D_n)$	...	$C(w_m, D_n)$

The model was originally developed for information retrieval in order to compute the similarity between two documents or between a document and a query.

Similarity is measured by the dot product of the corresponding vectors.



The Vector Space Model: Example

A collection of two documents:

D1: Chrysler plans new investments in Latin America.

D2: Chrysler plans major investments in Mexico.

The corresponding vectors:

D.	america	chrysler	in	investments	latin	major	mexico	new	plans
1	1	1	1	1	1	0	0	1	1
2	0	1	1	1	0	1	1	0	1

In the vector space model, documents are treated as bags of words (BOW); word order is ignored.

The dot product is $\mathbf{D1} \cdot \mathbf{D2} = 4$.

The cosine similarity is

$$\frac{\mathbf{D1} \cdot \mathbf{D2}}{\|\mathbf{D1}\| \cdot \|\mathbf{D2}\|} = \frac{4}{\sqrt{7} \cdot \sqrt{6}} \approx 0.62.$$



Giving a Weight

Word clouds provide a visual representation of word weights.



Image courtesy of Jonas Wisbrant.



$TF \times IDF$

Raw term frequency can be misleading.
Document coordinates are therefore usually computed as $tf \times idf$ (term frequency $\times$ inverse document frequency):

- Term frequency $tf_{i,j}$: the frequency of term j in document i
- Inverse document frequency:

$$idf_j = \log \frac{N}{n_j},$$

where N is the total number of documents in the collection and n_j is the number of documents that contain term j.

Note that several variants of tf–idf exist; see `https://en.wikipedia.org/wiki/Tf%E2%80%93idf`.



Document Similarity

Documents can be represented as vectors whose coordinates are the counts of each word:

$$\mathbf{d} = \big(C(w_1), C(w_2), C(w_3), \ldots, C(w_n)\big).$$

The similarity between two documents (or between a query $\mathbf{q}$ and a document $\mathbf{d}$) is given by their cosine:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\sum_{i=1}^{n} q_i d_i}{\sqrt{\sum_{i=1}^{n} q_i^2} \cdot \sqrt{\sum_{i=1}^{n} d_i^2}}.$$

The value ranges from 0 (orthogonal vectors) to 1 (identical or very similar vectors).



Categorizing Text

Text categorization (or classification) consists in assigning one or more classes to a text.
The text may range from a few words to an entire book.
In sentiment analysis (or opinion mining) the goal is to classify judgments or emotions expressed, for example, in product reviews into three basic categories:

- positive,
- negative,
- or neutral.

In spam detection the classifier assigns electronic messages to one of two classes:

- *spam*, or
- *ham* (non-spam).



The Reuters Corpus

The Reuters corpus contains approximately 800,000 English economic newswires and about 500,000 newswires in 13 other languages.
Each newswire is manually annotated with one or more topics chosen from a set of 103 predefined categories, for example:

C11: STRATEGY/PLANS

C12: LEGAL/JUDICIAL

C13: REGULATION/POLICY

C14: SHARE LISTINGS

etc.



Automatic Categorization

- Given an annotated corpus, supervised machine-learning techniques can be used to train classifiers.
- The training procedure represents each document as a bag of words, using Boolean features, term frequencies, or $tf \times idf$, together with the associated class labels.
- This representation step is called vectorization.
- Logistic regression is a simple yet effective technique for text classification.



Modeling the Texts

- *tf–idf* is an efficient vectorization method.
- Each text is stored as a row vector in a matrix X.
- The corresponding class labels are stored in a vector $\mathbf{y}$.

$$X = \begin{bmatrix} \text{text}_1 \\ \text{text}_2 \\ \text{text}_3 \\ \vdots \\ \text{text}_n \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} \text{class}_1 \\ \text{class}_1 \\ \text{class}_2 \\ \vdots \\ \text{class}_1 \end{bmatrix}.$$

A model is then trained to categorize the texts:

- In scikit-learn the training method is called `fit()`.
- Predictions are obtained with the method `predict()`.



Demo

Text categorization:
https://github.com/pnugues/pnlp/tree/main/notebooks

